OS and Parallelism

Slides by Raed Almomani and Andrew DeOrio

Agenda

- Motivation
- Processes
- Threads
- Synchronization
  - Atomic operations
- Summary

Running Multiple Tasks

- How to run multiple things at the same time?
  - A web server may need to handle multiple user requests at the same time (e.g., many users loading a webpage simultaneously).
- Threads and processes for parallelization
  - Today
- Network for communication
  - Next Time

Not parallel: Sequential program

- The Flask development server is a sequential program
- Handles one request at a time

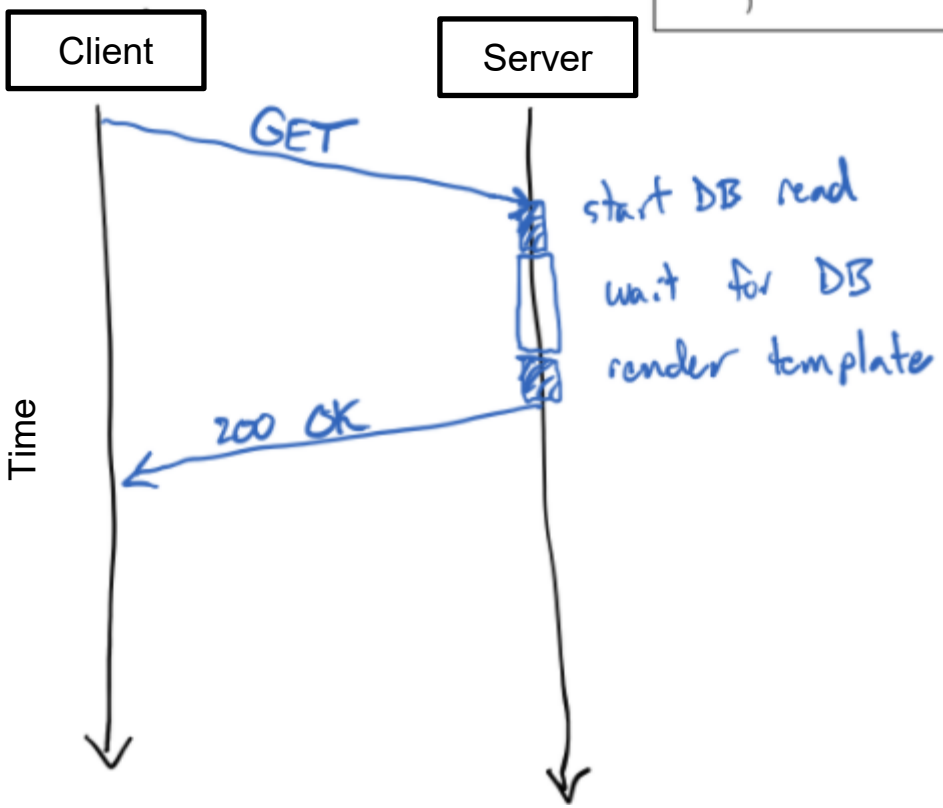
```
@app.route("/")
def index():
    context = # slow database read
    return flask.render_template("index.html", **context)
```

- First, we'll look at a one request from one client

```
$ curl localhost:5000
<html><body>Hello World!</body></html>
```

One request

```
@app.route("/")
def index():
    context = # slow database read
    return flask.render_template(
        "index.html", **context
    )
```



Note: Server must finish the database read before responding.

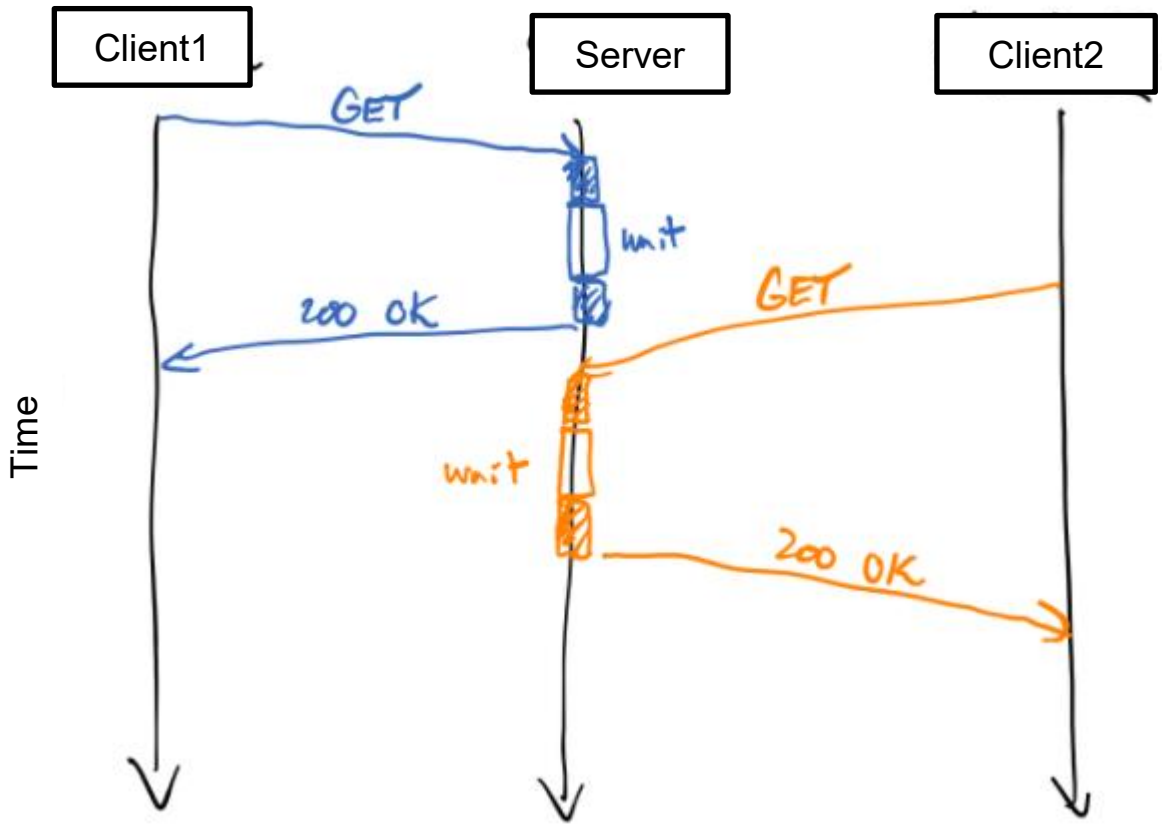
Problem: Server is idle while waiting for the database

Serial requests

- Two serial requests from two clients

- Request from client 1
- Start client 1 database read
- Wait for database
- Render client 1 template
- Response to client 1
- Request from client 2
- Start client 2 database read
- Wait for database
- Render client 2 template
- Response to client 2

Serial requests

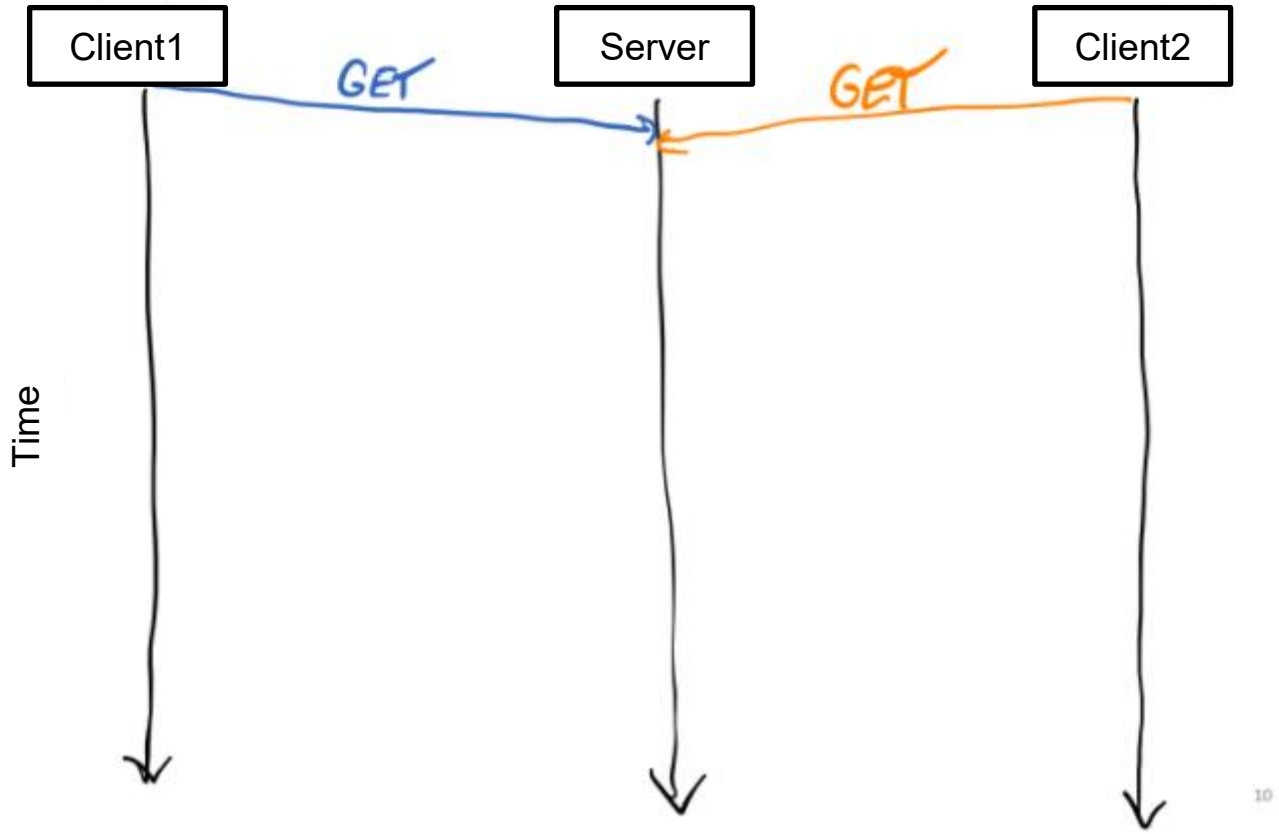


Concurrent requests, sequential program

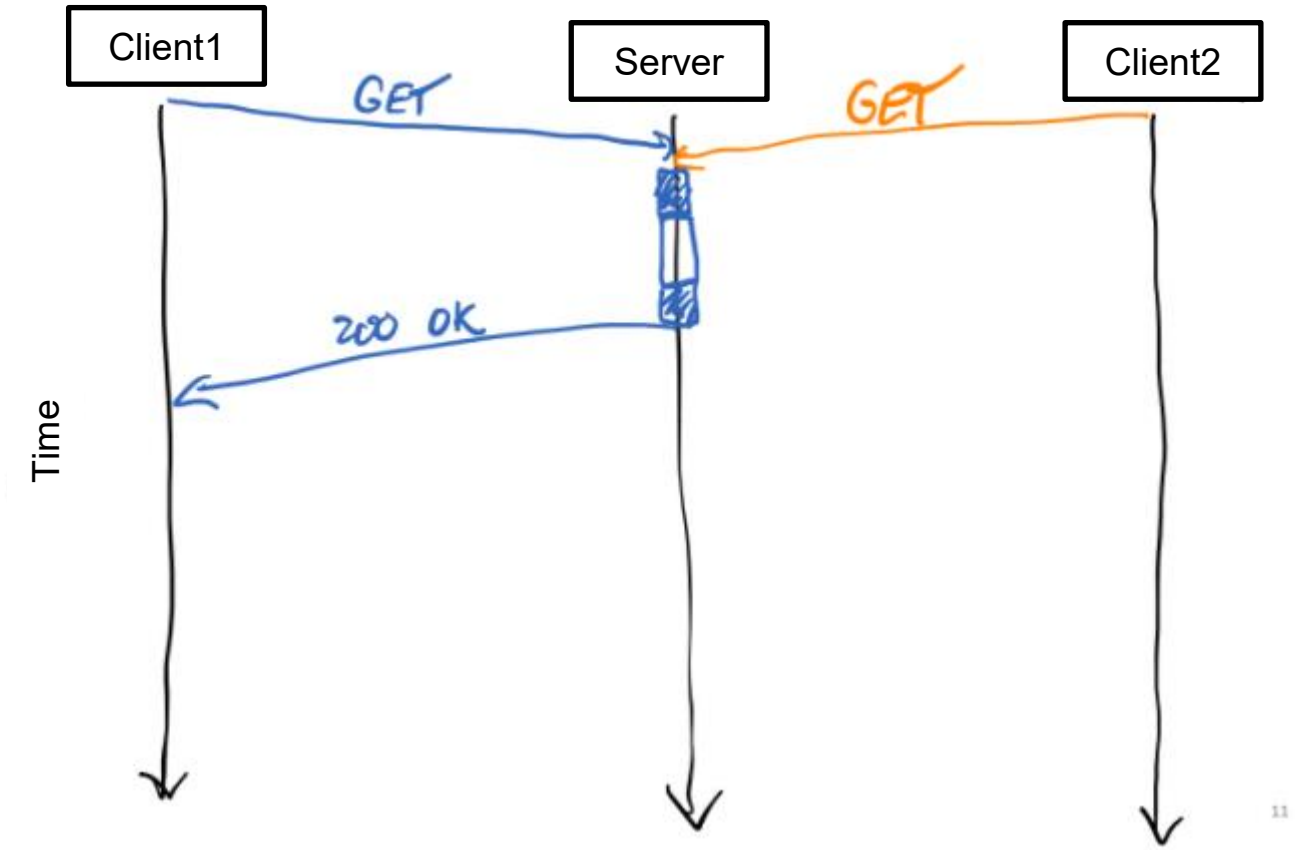
- Two concurrent requests from two clients handled by a sequential program

- Request from client 1
- Request from client 2
- Start client 1 database read
- Wait for database
- Render client 1 template
- Response to client 1
- Start client 2 database read
- Wait for database
- Render client 2 template
- Response to client 2

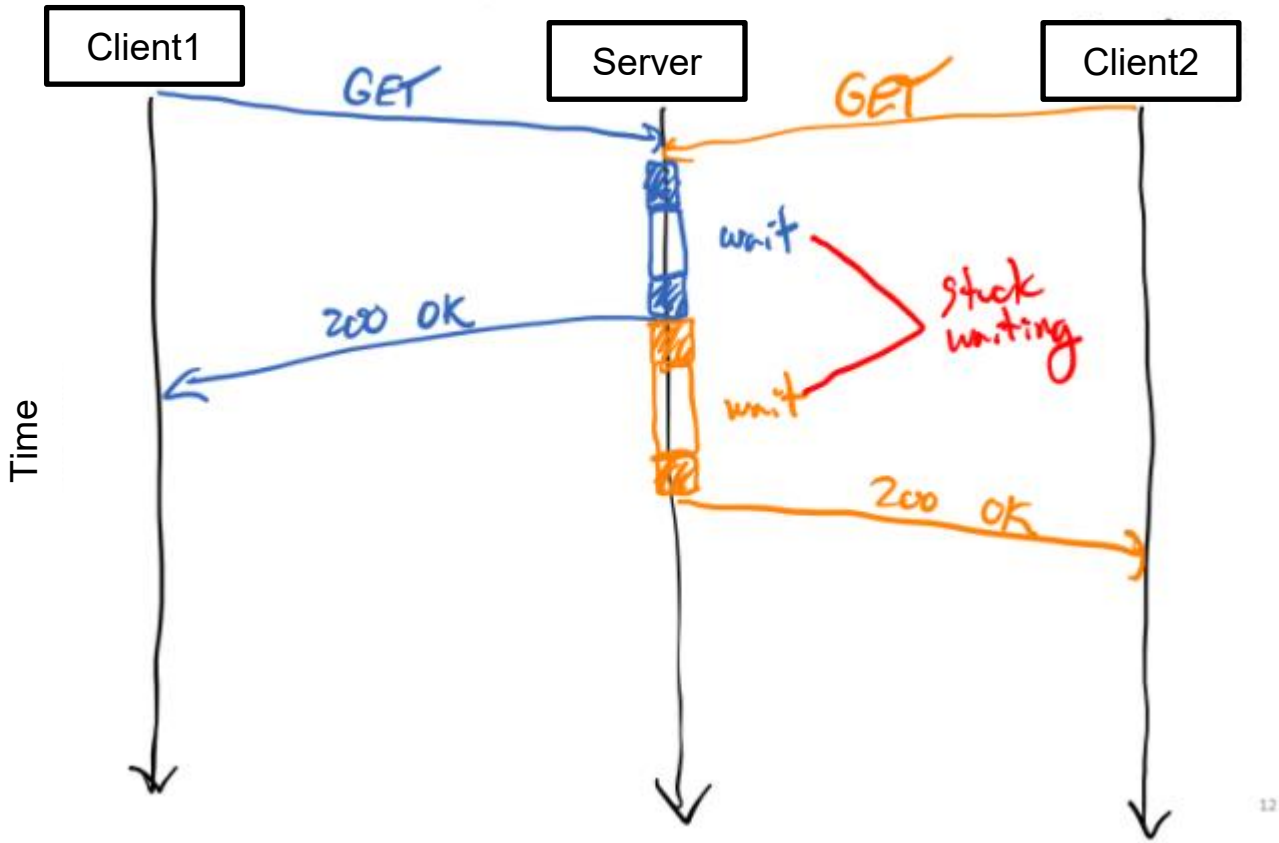
Concurrent requests, sequential program



Concurrent requests, sequential program



Concurrent requests, sequential program



Problem with sequential programs

- Problem with sequential programs when:
- Multiple things happening at once
  - Two concurrent requests in this example
- Slow resource
  - Database read in this example

- Our program spends a lot of time waiting
- Solution: Work on both requests at the same time

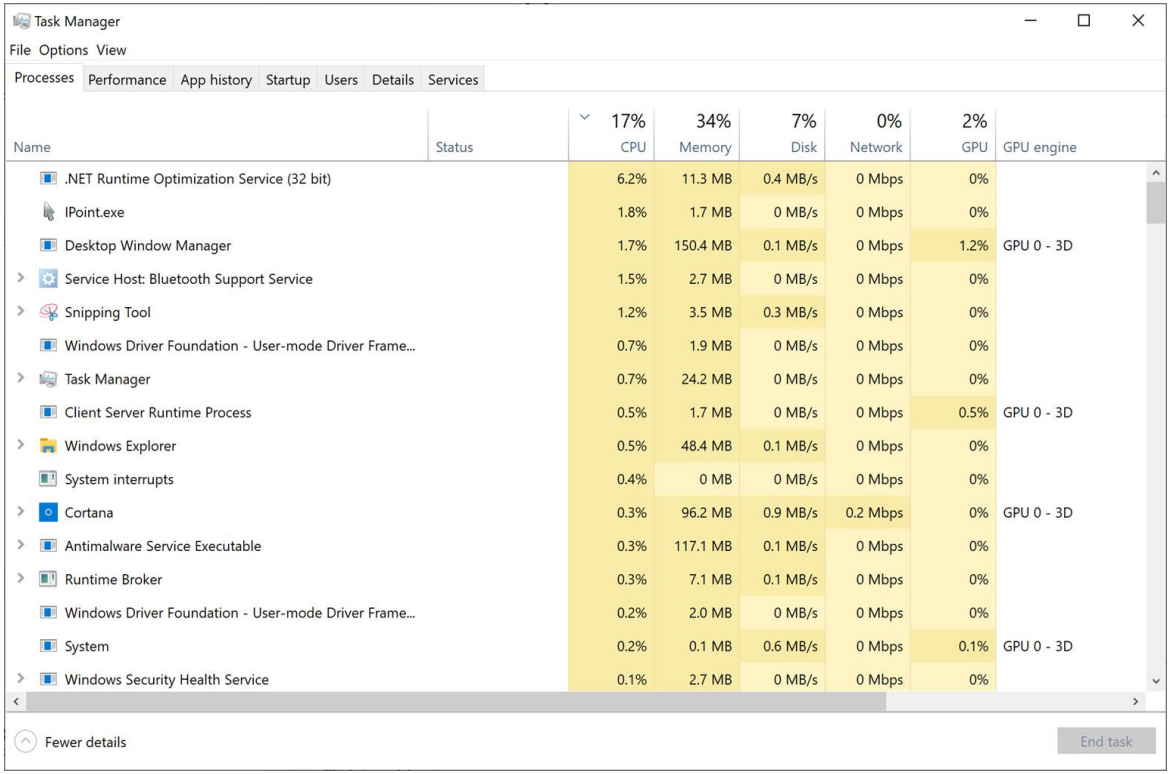
- Later today, we'll solve this problem with threads, but first we need to learn about processes

Agenda

- Motivation
- **Processes**
- Threads
- Synchronization
  - Atomic operations
- Summary

Processes

- A process is a running program



Name	Status	17% CPU	34% Memory	7% Disk	0% Network	2% GPU	GPU engine
.NET Runtime Optimization Service (32 bit)		6.2%	11.3 MB	0.4 MB/s	0 Mbps	0%	
IPoint.exe		1.8%	1.7 MB	0 MB/s	0 Mbps	0%	
Desktop Window Manager		1.7%	150.4 MB	0.1 MB/s	0 Mbps	1.2%	GPU 0 - 3D
Service Host: Bluetooth Support Service		1.5%	2.7 MB	0 MB/s	0 Mbps	0%	
Snipping Tool		1.2%	3.5 MB	0.3 MB/s	0 Mbps	0%	
Windows Driver Foundation - User-mode Driver Frame...		0.7%	1.9 MB	0 MB/s	0 Mbps	0%	
Task Manager		0.7%	24.2 MB	0 MB/s	0 Mbps	0%	
Client Server Runtime Process		0.5%	1.7 MB	0 MB/s	0 Mbps	0.5%	GPU 0 - 3D
Windows Explorer		0.5%	48.4 MB	0.1 MB/s	0 Mbps	0%	
System interrupts		0.4%	0 MB	0 MB/s	0 Mbps	0%	
Cortana		0.3%	96.2 MB	0.9 MB/s	0.2 Mbps	0%	GPU 0 - 3D
Antimalware Service Executable		0.3%	117.1 MB	0.1 MB/s	0 Mbps	0%	
Runtime Broker		0.3%	7.1 MB	0.1 MB/s	0 Mbps	0%	
Windows Driver Foundation - User-mode Driver Frame...		0.2%	2.0 MB	0 MB/s	0 Mbps	0%	
System		0.2%	0.1 MB	0.6 MB/s	0 Mbps	0.1%	GPU 0 - 3D
Windows Security Health Service		0.1%	2.7 MB	0 MB/s	0 Mbps	0%	

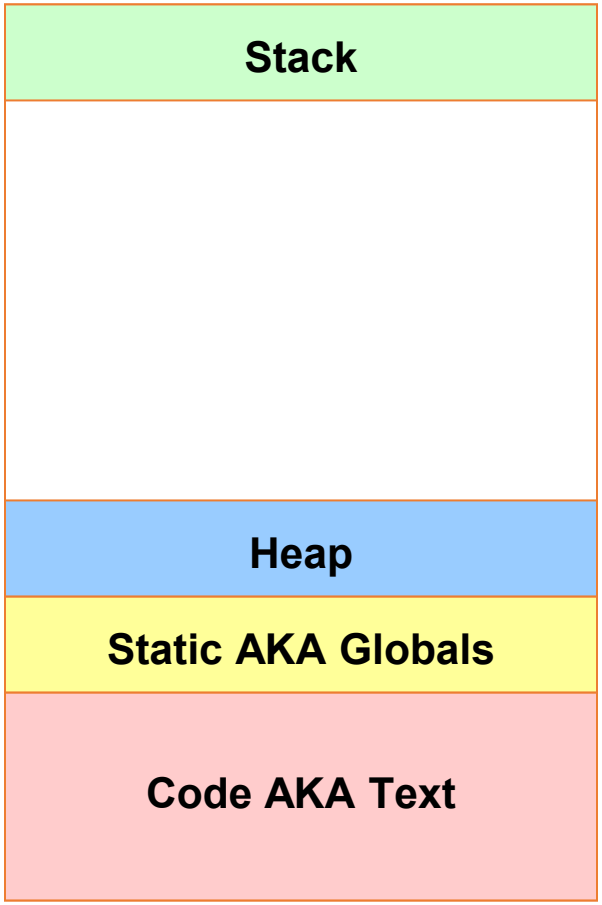
The process abstraction

- The *Process* is an OS abstraction for execution
  - Also sometimes called a job or a task
- A *process* is a program in execution
  - Programs are static entities with potential for execution

- Process consists of:
  - A unique process ID: *PID*
  - An address space: *Memory*
  - One or more threads: Sequences of computation
  - Some other resources: File handles, open sockets, etc.

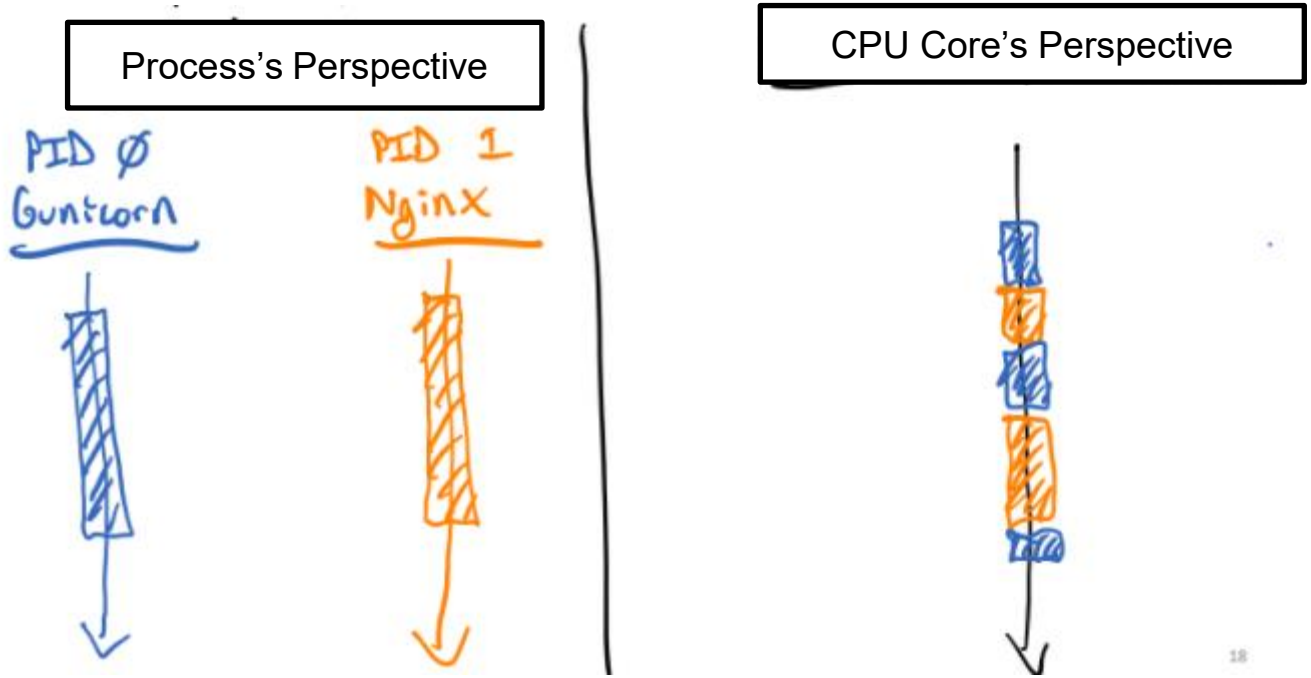
Process address space

- Each process gets its own address space
- Process can only write to its own address space



Operating system process scheduling

- Each CPU core can run one process at a time
- Every ~1-10ms, the OS can switch



Process abstraction

- On a single-core machine
  - Processes take turns
- On a multi-core machine
  - Processes run in parallel
  - Processes still take turns because there are usually many more processes than cores

- Recall that the *Process* is an OS abstraction for execution
- A process is an illusion of parallelism provided by OS

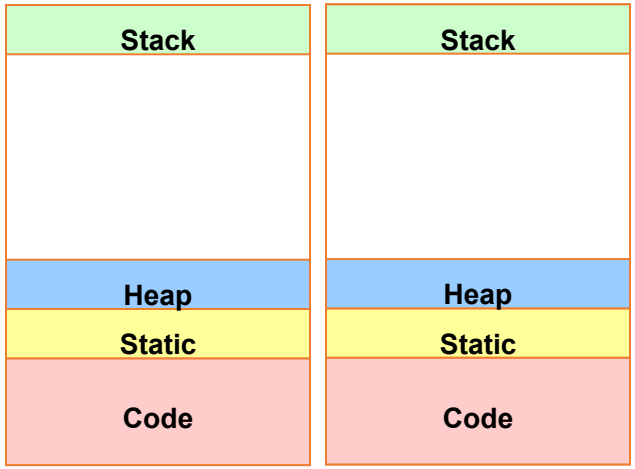
When are processes useful?

- Processes are most useful when:

- Multiple things happening at once
- Different programs running on the same machine
- Same program running on different machines

When are processes NOT useful?

- Back to our problem with sequential programs
  - Two concurrent requests with slow database operations
- Two processes can handle two requests concurrently
- Problem: Higher memory overhead
  - Copy of entire address space
  - They don't really need separate code, etc.

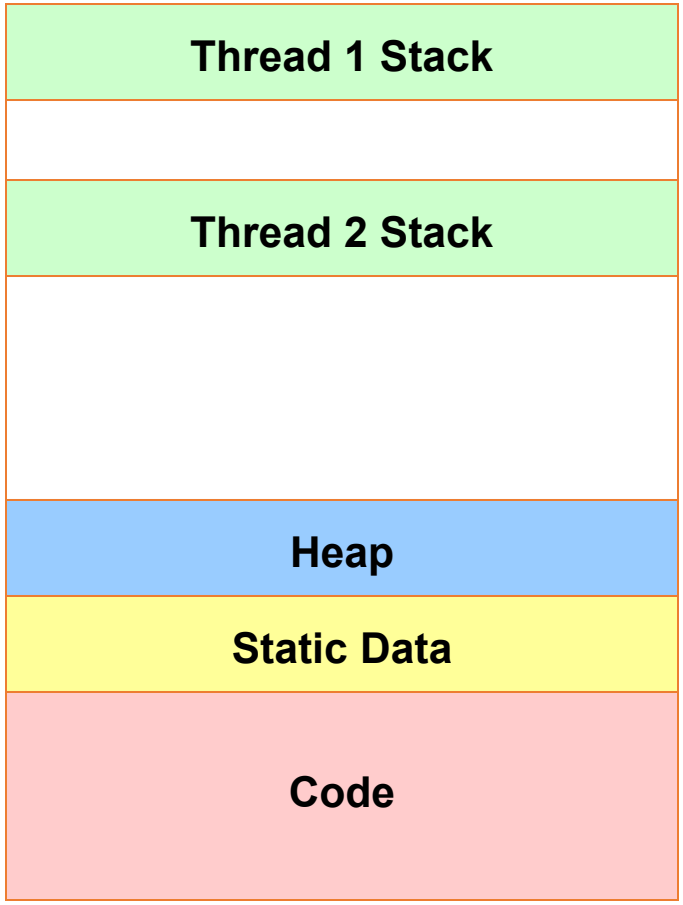


Agenda

- Motivation
- Processes
- **Threads**
- Synchronization
  - Atomic operations
- Summary

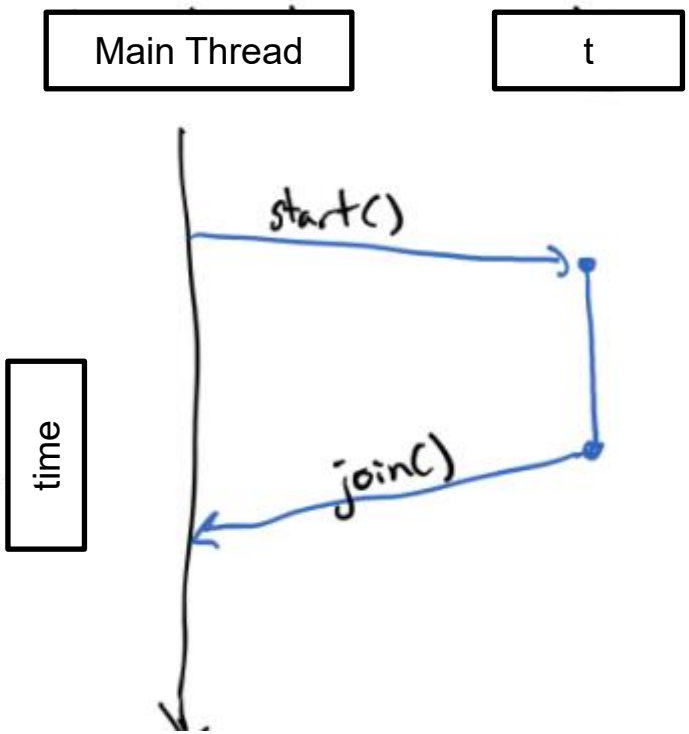
Threads

- *Threads* are multiple functions in one process running simultaneously
- Separate stacks
- Share heap, static data, code
- Lower overhead than processes



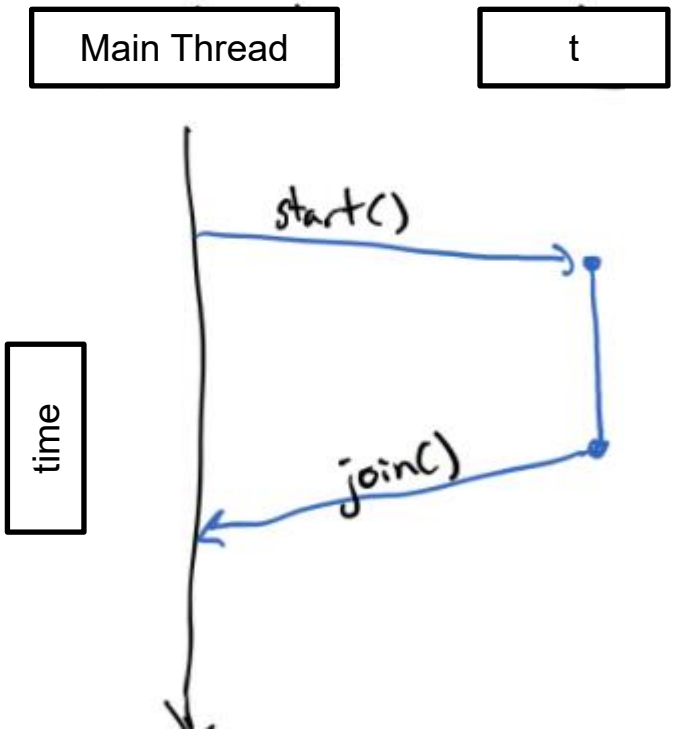
Thread example

```
from threading import Thread

def hello():
    print("hello world")

t = Thread(target=hello)
t.start()
t.join()
```



Thread example

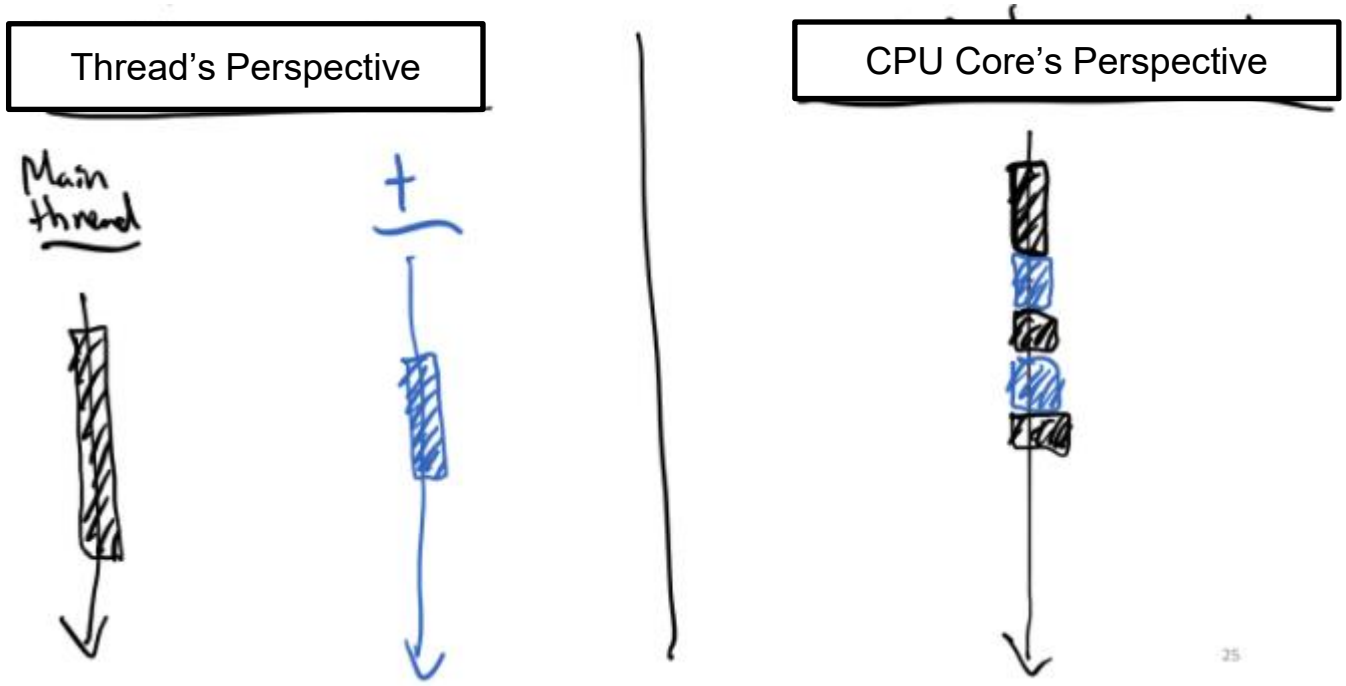
```
from threading import Thread

def hello():
    print("hello world")

t = Thread(target=hello)
t.start()
t.join()
```



- `t.join()` waits for `hello()` to return
- `t.join()` does not interrupt `hello()`

Operating system thread scheduling

- Each CPU core can run one thread at a time
- Every ~1-10ms, the OS can switch



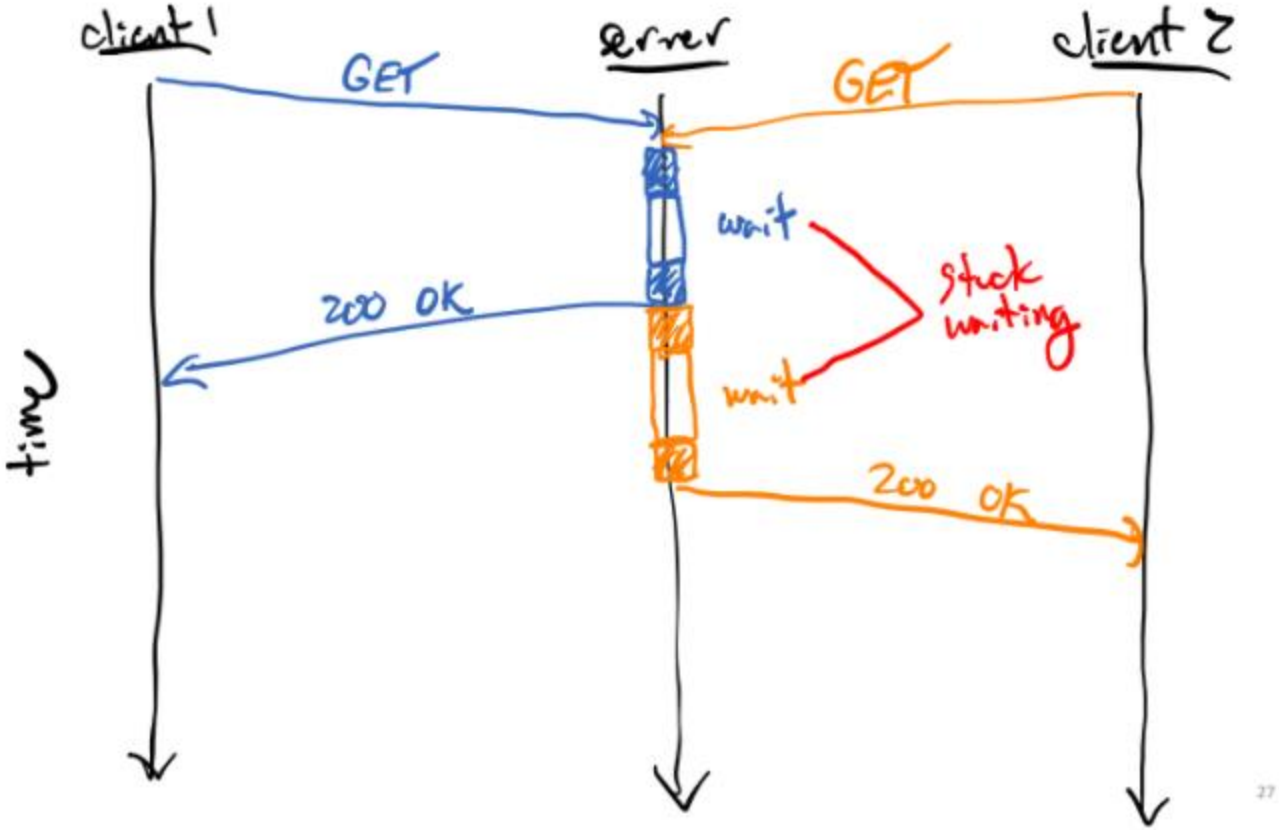
Thread abstraction

- On a single-core machine
  - Threads take turns
- On a multi-core machine
  - Threads run in parallel
  - Threads still take turns because there are usually more threads than cores

- Threads have a lot in common with Processes
  - Both provide an illusion of parallelism
  - Both can provide true parallelism on the right hardware

- Threads share memory
  - Less overhead
  - Possible security risk, depending on the application

Recall: Concurrent requests with 1 thread



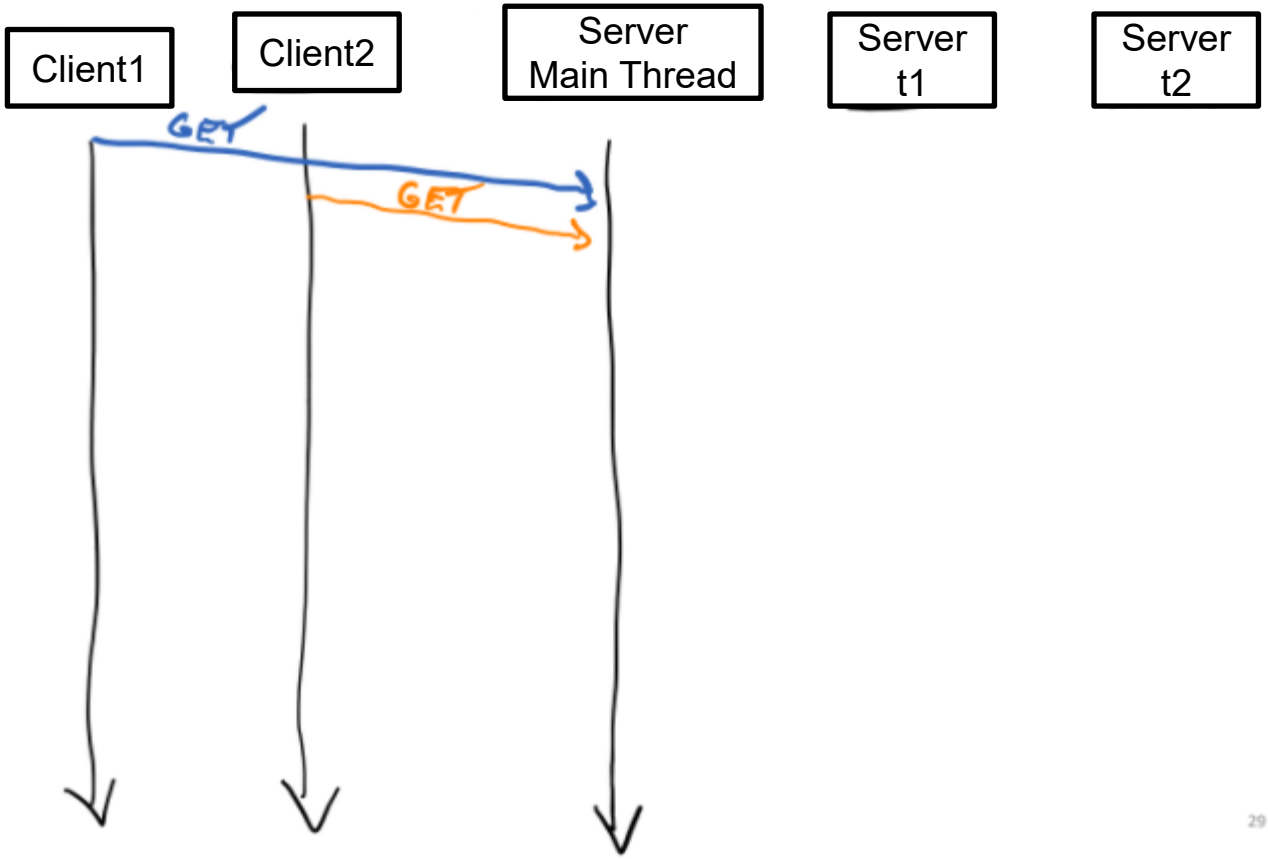
Concurrent requests with multiple threads

- Two concurrent requests from two clients to server with two threads

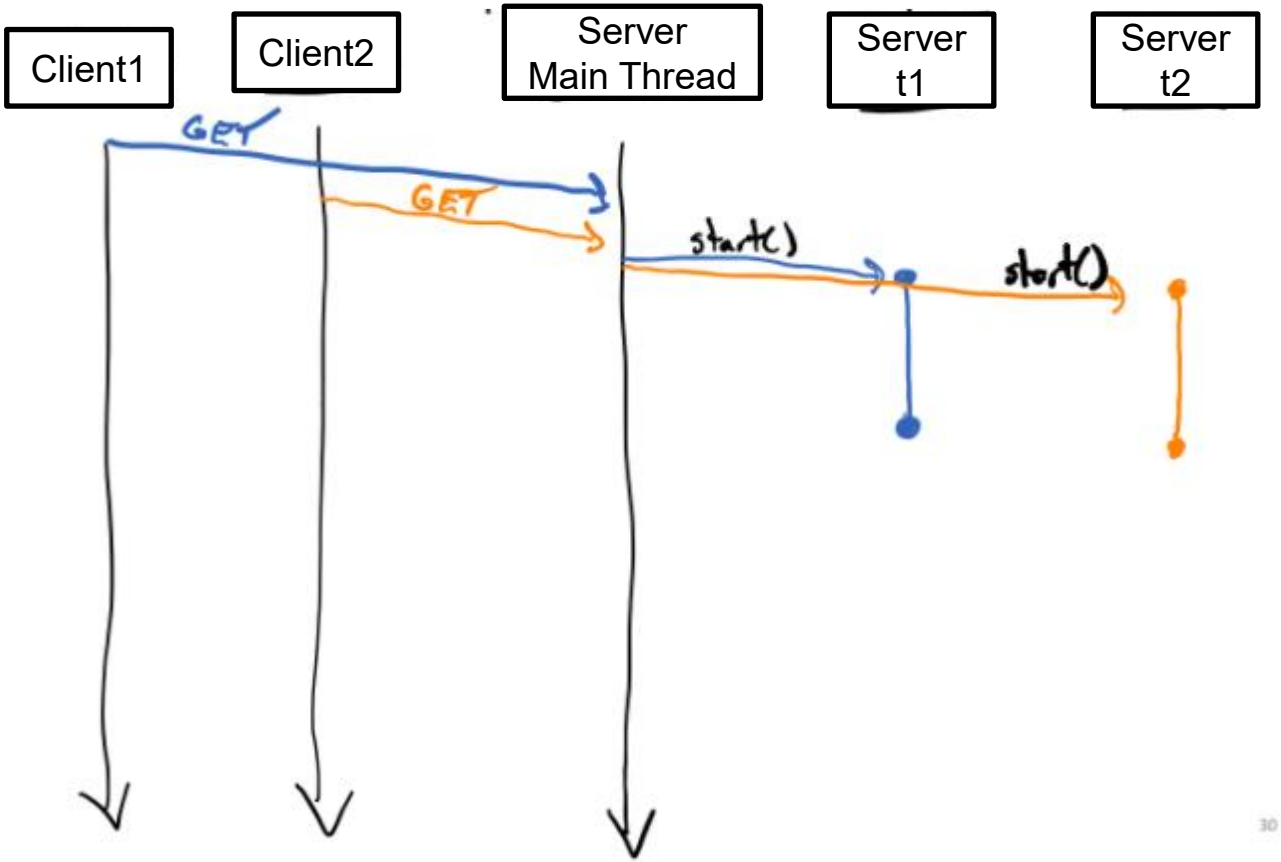
- Request from client 1, assign to thread 1
- Request from client 2, assign to thread 2
- Start client 1 database read
- Start client 2 database read
- Wait for client 1 database read
- Wait for client 2 database read
- Render client 1 template
- Response to client 1
- Render client 2 template
- Response to client 2

Note: Other interleavings are possible

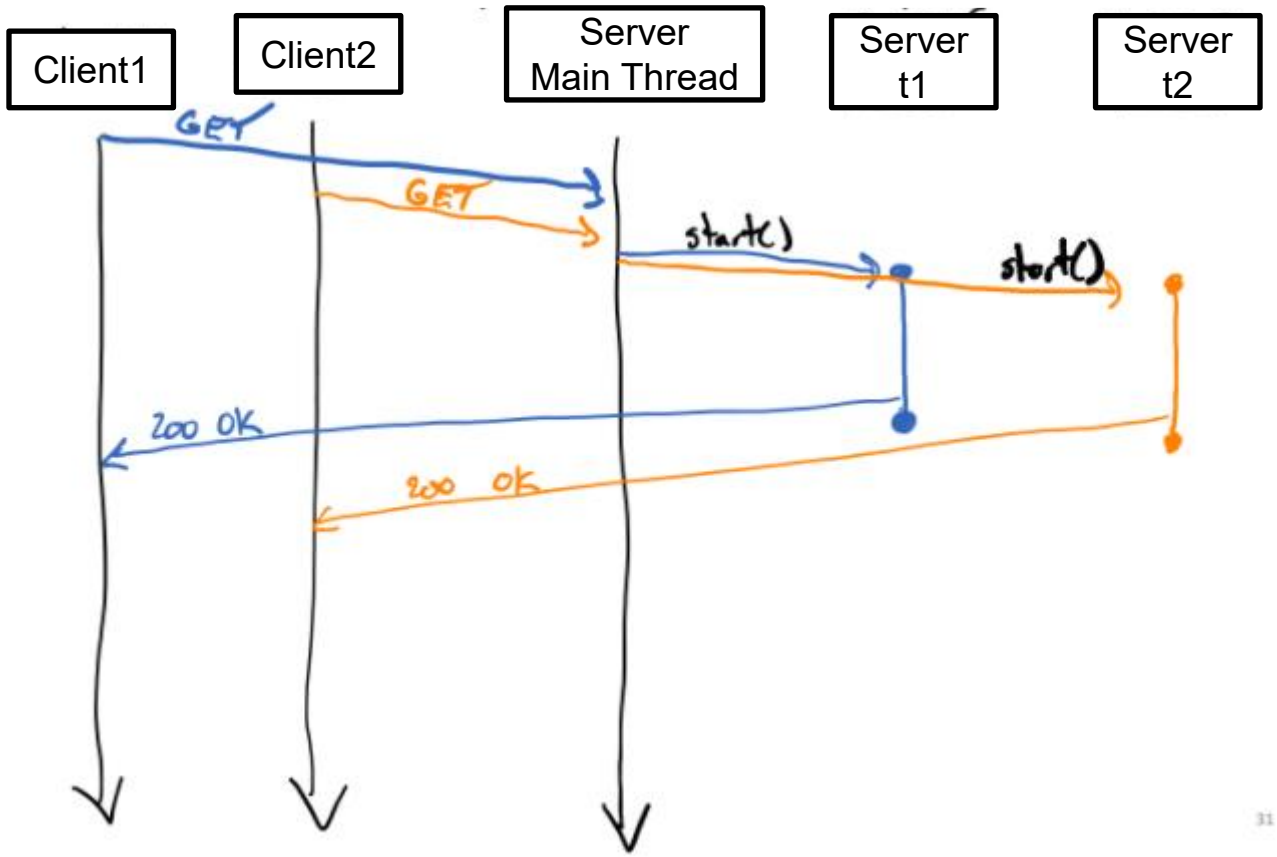
Concurrent requests with multiple threads



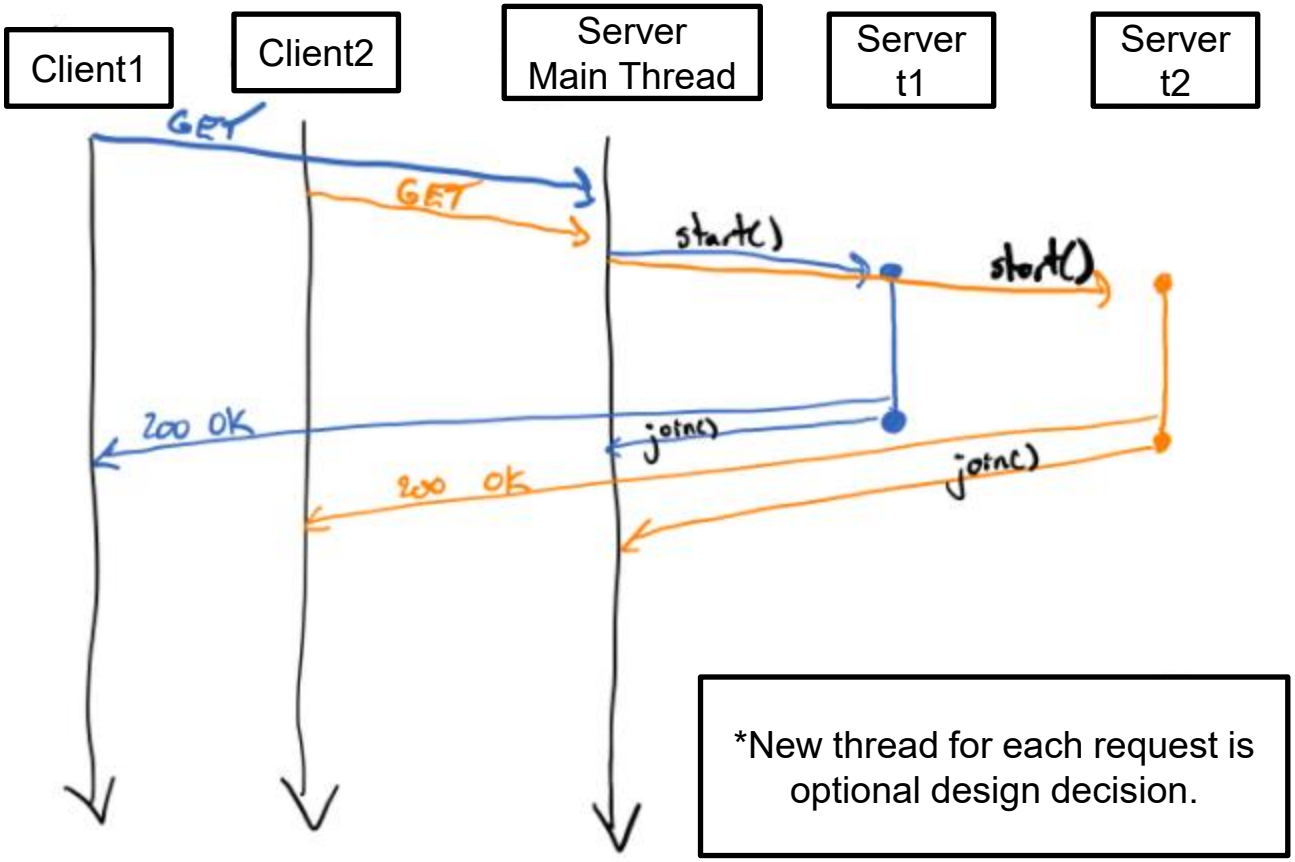
Concurrent requests with multiple threads



Concurrent requests with multiple threads



Concurrent requests with multiple threads



When are threads useful?

- Multiple things happening at once
  - Within one program
- May need to share data in memory
- Usually some slow resource
  - Network response
  - Disk I/O
  - Reading a sensor
  - Human input

Benefits of threads

- Simpler programming model
  - The illusion of a dedicated CPU per thread
  - State for each thread (local variables)

- OS takes care of CPU sharing
  - Other threads can progress while one thread waits for I/O

Processes vs. threads

- **Processes** have separate address spaces
  - Useful when there is not complete trust
  - Limit the damage caused by buggy code
  - Required to run different programs on the same computer
  - Required to run the same program on different computers

- **Threads** share an address space
  - Lower memory overhead
  - Faster context switches (threads taking turns)
  - Can share data

- Both implement concurrency
- Both are illusions of parallelism
- Both can provide true parallelism with the right hardware

Agenda

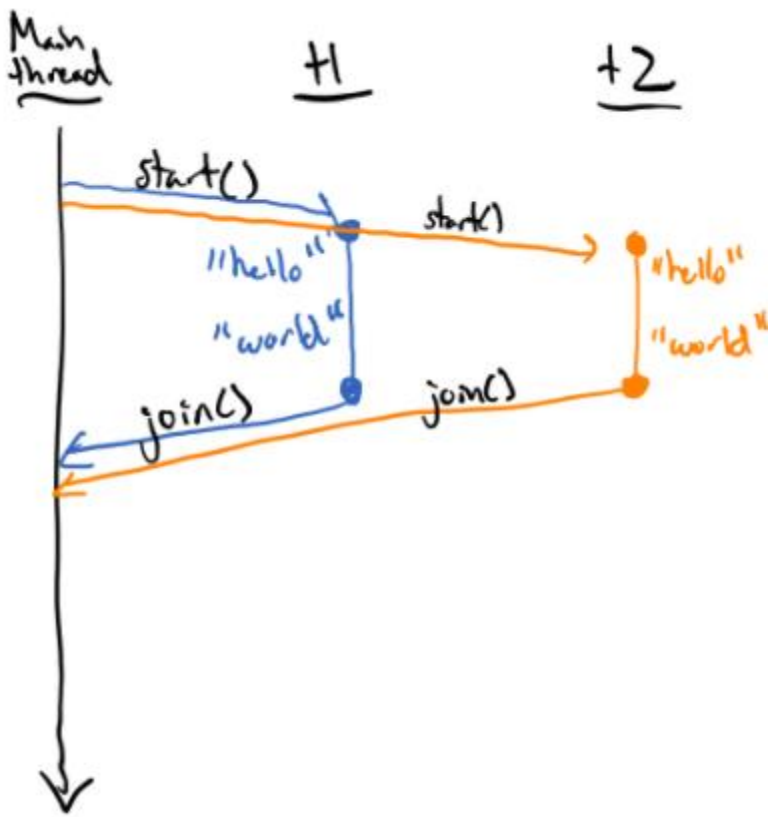
- Motivation
- Processes
- Threads
- **Synchronization**
  - Atomic operations
- Summary

Cooperating threads

- What happens when threads cooperate?
- What are these threads sharing? The output stream.
- What is the output of this code?

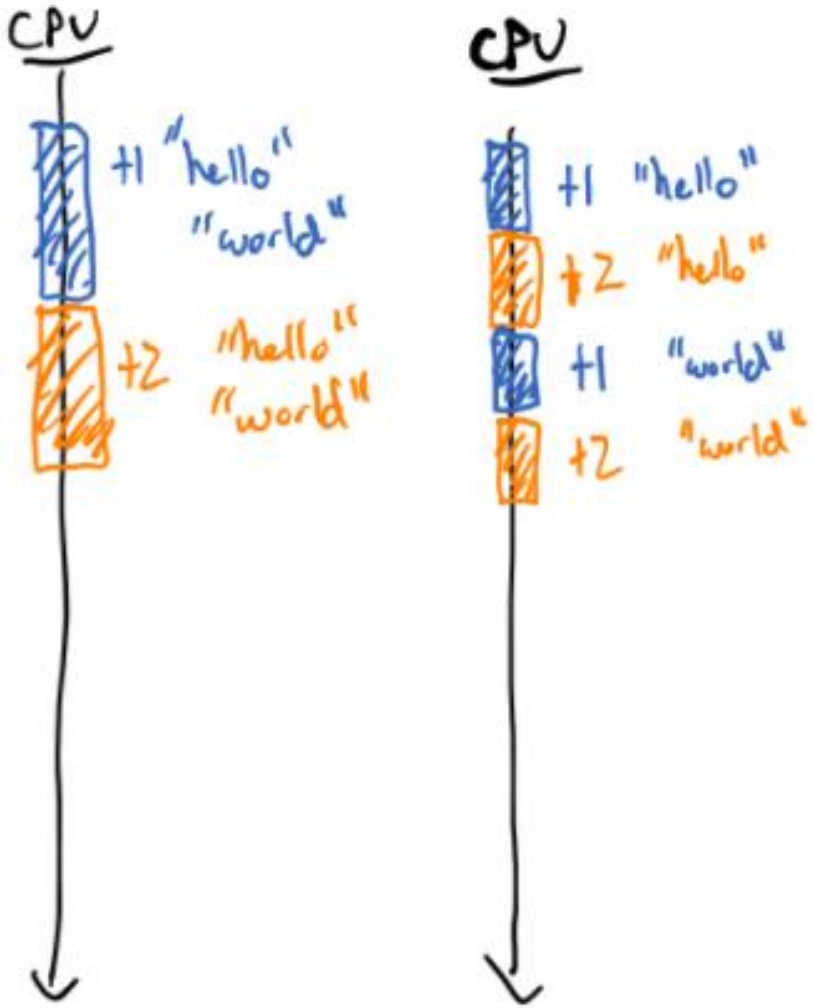
```
def hello():
    print("hello")
    print("world")

t1 = Thread(target=hello)
t2 = Thread(target=hello)
t1.start()
t2.start()

t1.join()
t2.join()
```

Cooperating threads




```
def hello():
    print("hello")
    print("world")

t1 = Thread(target=hello)
t2 = Thread(target=hello)
t1.start()
t2.start()

t1.join()
t2.join()
```

Cooperating threads




```
def hello():
    print("hello")
    print("world")

t1 = Thread(target=hello)
t2 = Thread(target=hello)
t1.start()
t2.start()

t1.join()
t2.join()
```

Multiple ways to interleave the operations from t1 and t2

Many possible outputs

```
def hello():
    print("hello")
    print("world")

t1 = Thread(target=hello)
t2 = Thread(target=hello)
t1.start()
t2.start()
```

- Possible outputs

```
hello
world
hello
world
```

```
hello
hello
world
world
```

- Impossible outputs

```
world
hello
world
hello
```

```
world
world
hello
hello
```

Unpredictable order

- When threads share data, the output can be unpredictable
- OS decides when threads take turns
- Different timing -> different thread order - > different output

Shared data

```
def hello():
    print("hello")
    print("world")

t1 = Thread(target=hello)
t2 = Thread(target=hello)
t1.start()
t2.start()

t1.join()
t2.join()
```

- Insight: Shared data
- The output buffer in this example

- Ordering within a thread is sequential
- Many ways to merge multiple threads together into global order

- Problem: Some global orderings may produce incorrect results
- Solution: Locks

Agenda

- Motivation
- Processes
- Threads
- Synchronization
  - **Atomic operations**
- Summary

Atomic operations

- An *atomic* operation appears to the other threads as if it happened instantaneously
  - No code from other threads can occur in between

- In our previous example, `print()` behaved atomically*

*The `print()` output was line buffered, which means different lines of output cannot be interleaved (more detail).

Locks

- How can we make sure "hello world" is printed by each thread?
- Limit when threads can take turns

- A *Lock* is for mutual exclusion
  - Also called a *mutex*

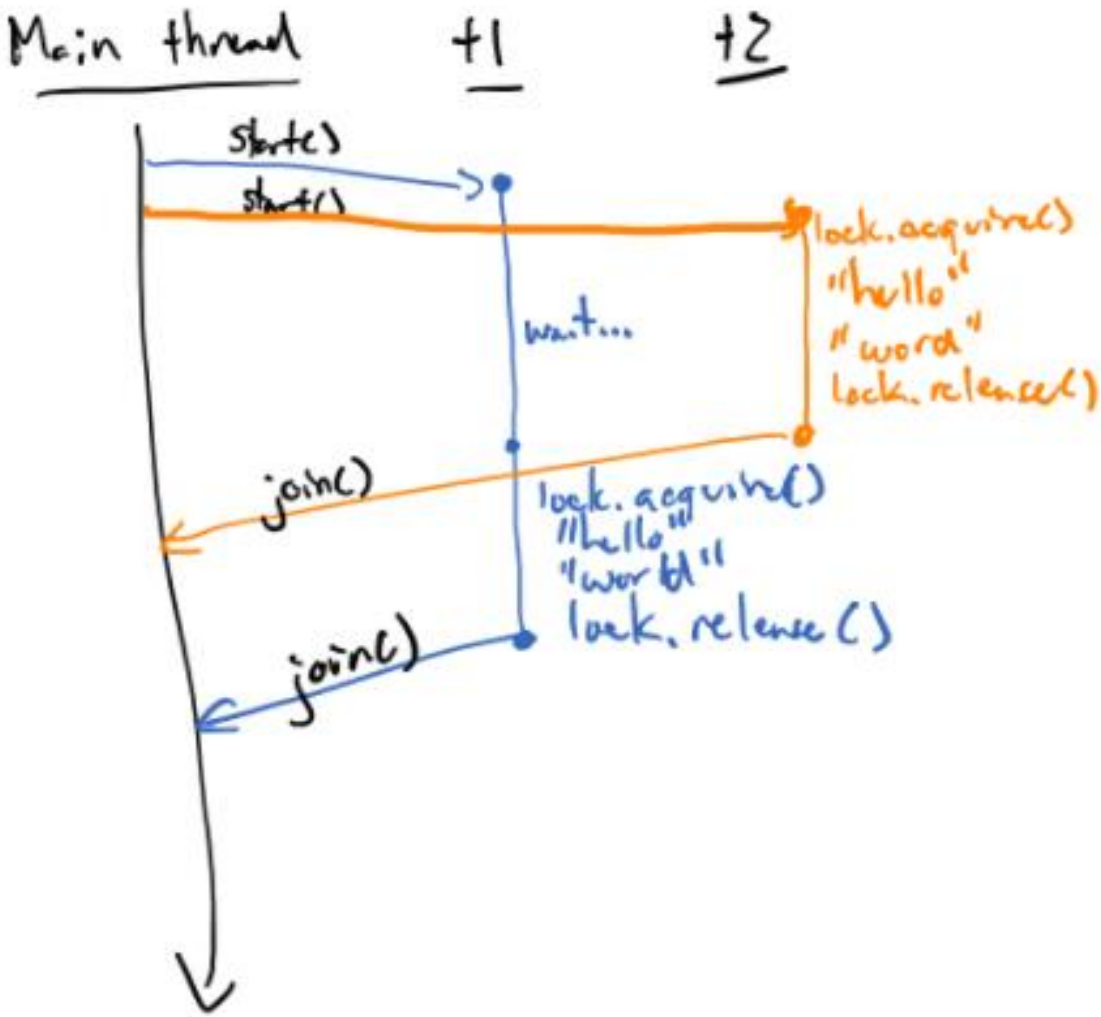
- A *Lock* lets the programmer limit when threads take turns

Locks

- How can we make sure "hello world" is printed by each thread?
- Limit when threads can take turns

```
import time
import random
from threading import current_thread,
Thread, Lock

def hello(lock):
    name = current_thread().name
    lock.acquire()
    print("{name} hello")
    print("{name} world")
    lock.release()

lock = Lock()
t1 = Thread(target=hello, args=(lock,))
t2 = Thread(target=hello, args=(lock,))
t1.start()
t2.start()

t1.join()
t2.join()
```

Locks


Main thread
t1
t2
start()
start()
lock.acquire()
"hello"
"word"
lock.release()
wait...
join()
lock.acquire()
"hello"
"world"
lock.release()
join()

Agenda

- Motivation
- Processes
- Threads
- Synchronization
  - Atomic operations
- **Summary**

Next time

- How to run multiple things at the same time?
- Threads and processes for parallelization
  - Today
- Networking for communication
  - Next time

Summary

- Parallelization is important for implementing distributed systems
- **Processes** and **threads** are used to write parallel programs
- With parallel programs, need to think about **synchronization** of data
- To synchronize data, we use **atomic operations**